

SYSTEMD

Helix Platform — systemd Boot Integration

Revision: 2 **Last modified:** 2026-07-28 **Maintainer:** HelixCode platform

The whole Helix platform — HelixCode, HelixAgent, HelixLLM and their infrastructure — is installed as systemd **user** units that start on host boot and survive restarts.

Quick reference

```
./setup.sh # install everything
           (units included)
./setup.sh --start # ... and start it
           now
```

```
systemctl --user start helix.target # bring the whole
           platform up
systemctl --user stop helix.target # bring it all down
systemctl --user status helixagent # one service
systemctl --user list-units 'helix*' # all of them
journalctl --user -u helixagent -f # follow one
           service's logs
```

```
scripts/install_systemd_units.sh # re-install after
           editing a unit
scripts/install_systemd_units.sh --start # install + enable +
           start
scripts/install_systemd_units.sh --uninstall
```

Prove the platform is actually *serving*, not merely active (§11.4.108):

```
curl -s http://localhost:8100/api/scores #
           llmsverifier
curl -sk https://localhost:8443/internal/health # helixllm-
           gateway (TLS)
curl -s http://localhost:7061/health # helixagent
curl -s http://localhost:8081/health # helixcode-
           server
```

Units

`helix.target` is the umbrella; each service declares `WantedBy=helix.target`, so enabling a service wires it in automatically.

Unit	Type	Port	Purpose
<code>helix.target</code>	target	—	Umbrella — start/stop the platform as one
<code>helixcode-infra.service</code>	oneshot	see below	11 containers via podman compose — Postgres, Redis, memcached, Ollama, Selenium, chromedp, Weaviate, ChromaDB, Qdrant, Cognee, plus a multicast-router sidecar that publishes no host port
<code>llmsverifier.service</code>	simple	8100	LLMsVerifier — model/provider verification + scoring API
<code>helixllm-coder.service</code>	oneshot	18434	Local Qwen3-Coder-30B model container
<code>helixllm-gateway.service</code>	simple	8443 (TLS)	HelixLLM multi-provider LLM router
<code>helixagent.service</code>	simple	7061	HelixAgent runtime (HTTP/1.1+2 on TCP, HTTP/3 on UDP)
	simple	8081	

Unit	Type	Port	Purpose
helixcode-server.service			HelixCode API server

Start ordering: infra → llmsverifier → llm-coder / llm-gateway → helixagent → helixcode-server.

Dependency strength is deliberate per service:

- helixagent uses Requires=helixcode-infra.service — its startup dependency verification is mandatory and hard-fails, so starting without infra is pointless.
- helixllm-gateway uses Wants=llmsverifier.service — the gateway polls {verifier}/api/scores and falls back to a static score table when it is unreachable. That fallback is a real working mode, so a down verifier must not prevent the gateway from starting and routing.
- helixcode-server uses Wants= for the same reason: it has its own infra-boot fallback path.

Live state captured 2026-07-28 — a point-in-time observation, not a standing guarantee; re-probe before relying on it. All six units active, enabled, NRestarts=0; 14 containers running on the host, 11 healthy, 0 unhealthy; each of the four HTTP health routes in the Quick reference returned 200.

Port map

Port	Service	Notes
8081	helixcode-server	config/replica-8081.yaml
8443	helixllm-gateway	TLS — use https://. Health route is /internal/health (registered in submodules/helix_llm/internal/server/server.go:167). <i>Not</i> /v1/models — that is the path the gateway uses to probe downstream providers, and it answers {"object": "list", "data": null} until provider keys are configured, so it is a misleading health signal
7061	helixagent	TCP <i>and</i> UDP (QUIC)
8100	llmsverifier	the address the gateway looks for; the binary's own default is 8080, so

Port	Service	Notes
		the unit passes --port 8100 explicitly
18434	helixllm-coder	local model
5433	infra postgres	not 5432 — 5432 belongs to an unrelated stack on this host
6380	infra redis	not 6379, same reason
8083	infra weaviate	HTTP; moved off 8081, which helixcode-server owns
50051	infra weaviate	gRPC
8082	infra chromadb	
8000	infra cognee	
6333/6334	infra qdrant	
11434	infra ollama	
11211	infra memcached	
4444/7900	infra selenium	
9222	infra chromedp	
—	infra multicast-router	sidecar; container-network only, publishes no host port

Boot chain

These are **user** units, not system units, because the services run rootless podman (§11.4.161) as the invoking user and read that user's ~/.config/cdi GPU specs.

Boot therefore depends on three links, **all** of which must hold:

```
loginctl Linger=yes                                # user manager runs without a 1
└─ default.target.wants/helix.target                # helix.target enabled
    └─ helix.target.wants/*.service                 # each service enabled
```

1. **Linger.** User units normally start at *login*. `loginctl enable-linger $USER` is what makes them start at *boot* instead — the installer enables it and reports whether it succeeded. Without *linger* the platform only comes up once someone logs in. This is the link most often missing.
2. **helix.target enabled.** The target declares `WantedBy=default.target`, so enabling it creates the `default.target.wants/helix.target` symlink that pulls the platform in at boot.

3. **Each service enabled.** Every service declares `WantedBy=helix.target`, so `systemctl --user enable <unit>` creates the `helix.target.wants/<unit> symlink`. The target file itself enumerates no members — it is populated purely by these symlinks, which is why adding or removing a service never requires editing the target.

Enable / disable

```
# enable everything (what the installer does)
systemctl --user enable helix.target
systemctl --user enable helixcode-infra helixllm-coder helixllm-
gateway \
llmsverifier helixagent helixcode-server
loginctl enable-linger "$USER"
```

```
# disable one service – removes only its helix.target.wants/
symlink
systemctl --user disable helixcode-server
```

```
# stop the platform coming up at boot, leaving the units
installed
systemctl --user disable helix.target
```

`enable` only wires a unit into the boot graph; it does not start it. Add `--now`, or `systemctl --user start helix.target`, to do both.

Verify all three links:

```
loginctl show-user "$USER" -p Linger           # expect
Linger=yes
ls ~/.config/systemd/user/default.target.wants/ # expect
helix.target
ls ~/.config/systemd/user/helix.target.wants/
# expect the six services
systemctl --user is-enabled helix.target helixagent
```

Unit templating

The units in `scripts/systemd/` are **templates** containing `@HELIX_ROOT@` and `@HELIXLLM_BIN@`. `scripts/install_systemd_units.sh` expands them for the current checkout when installing into `~/.config/systemd/user/`.

Never hand-edit the installed copies — edit the template in `scripts/systemd/` and re-run the installer, otherwise the change is lost on the next install and a

fresh clone cannot reproduce it. The installer refuses to install a unit that still contains an unexpanded placeholder.

Secrets

helixagent needs DB_PASSWORD and JWT_SECRET; both are read from .env via EnvironmentFile= and are **never** inlined into unit files, which are world-readable and version-controlled (CONST-042 / §12.1). .env is mode 0600 and gitignored.

If .env is missing, EnvironmentFile=- (leading -) makes it optional, so the service fails with a clear dependency error rather than a unit-file parse error.

Gotchas

A unit env var the binary never reads fails silently

These binaries are env-configured through envconfig struct tags, not through a config flag. If a unit sets a variable name the struct does not declare, the setting is simply **inert** — no parse error, no warning, no failed unit. The service starts, reports active (running), and quietly runs on the compiled-in default. This is the worst failure shape in the whole file: everything is green and the intent never took effect.

Two instances of this class, both found and fixed on 2026-07-28 in helixllm-gateway.service:

Unit set	Binary actually reads	Consequence of the mismatch
HELIX_CACHE_REDIS_HOST / HELIX_CACHE_REDIS_PORT	HELIX_REDIS_HOST / HELIX_REDIS_PORT (config.go:121-122, port default 6379)	Redis fell back to :6379, where nothing listens
(unset)	HELIX_MODELS_DIR (config.go:70, default /models)	Default is a container path; running natively, every model auto-download failed

Symptoms the gateway logged on every start, before the fix:

```
dial tcp 127.0.0.1:6379: connect: connection refused
KV cache: Redis unreachable, falling back to in-memory
downloader: create models dir /models: mkdir /models: permission denied
```

Note that both fell back to a *working degraded mode* — in-memory cache, no downloads — which is precisely why they survived unnoticed.

Rule: when adding or changing an `Environment=` line, diff the name against the binary's actual env tags. Never infer the name from the config key.

```
# every env var the binary really reads
grep -rn 'env:"' submodules/helix_llm/internal/shared/config/
config.go
```

```
# every env var the unit sets
grep -o 'Environment=[A-Z_]*' scripts/systemd/helixllm-
gateway.service
```

Compiled-in defaults aimed at a container (`/models`, `:5432`, `:6379`, `:8001`) are a recurring source of this: they are plausible, they are wrong for a native user service, and they never announce themselves.

active does not mean *listening* — the gateway proves it

helixllm-gateway downloads models **before** it binds its listener. Between ExecStart and the first accepted connection the unit is already active (running) while `:8443` refuses connections. On a first boot after a model-cache wipe that window is **several minutes** long.

So a green `systemctl --user status` during that window is accurate and useless. Probe the port, not the unit state (§11.4.108):

```
curl -sk https://localhost:8443/internal/health # the real
readiness signal
journalctl --user -u helixllm-gateway -f # watch the
download progress
```

Raising `TimeoutStartSec` does not help here: the unit is `Type=simple`, so systemd considers it started the moment the process forks — it is not waiting on readiness at all.

podman ps is not a reachability oracle

helixcode-infra.service sets `KillMode=process`. podman compose up -d returns immediately but leaves common and rootlessport (the host-side port forwarders) in the unit's cgroup. Under the default `KillMode=control-group`, any stop or failure makes systemd SIGKILL the whole cgroup — killing the port

forwarders of every container in the stack while the containers themselves survive.

When that happens `podman ps` still reports Up (healthy) 0.0.0.0:5433->5432/tcp, because the container is alive; only the forwarder is gone. Nothing on the host can connect.

Always confirm reachability with `ss -ltn` or `curl`, never with `podman ps`.

```
# what is ACTUALLY reachable
for p in 5433 6380 8083 8082 8000 6333 11434; do
  ss -ltn | grep -q ":$p " && echo "ok $p" || echo "DEAD $p"
done
```

helixagent serves HTTP/3, so it binds UDP too

`ss -ltn` (TCP only) will not show the QUIC listener. Use `ss -ltn` and `ss -lun`, or `ss -an | grep 7061`.

Troubleshooting

<code>systemctl --user status <unit> -l</code>	<code># why it failed</code>
<code>journalctl --user -u <unit> -n 50</code>	<code># recent log</code>
<code>journalctl --user -u <unit> --since today</code>	<code># today's log</code>
<code>systemctl --user reset-failed</code>	<code># clear failed state</code>
<code>before retrying</code>	
<code>systemctl --user cat <unit></code>	<code># the installed</code>
<code>(expanded) unit</code>	

A unit is active (running) but nothing is listening. Unit state is not proof the service works (§11.4.108). Check the journal for where startup stalled, and probe the port directly. For `helixllm-gateway` this is expected behaviour during model download, not a fault — see *Gotchas*.

Infra fails with address already in use. Something else on the host holds the port. Identify the owner *before* acting — it may belong to another project (§11.4.174):

```
ss -ltnp | grep ':<port>'
```

Known gaps

Recorded honestly rather than papered over (§11.4.6):

- **distributed_locks table has no schema.** HelixAgent references it from three Go files but no migration or .sql file defines it, so it logs Error cleaning locks: ERROR: relation "distributed_locks" does not exist (SQLSTATE 42P01) every 60 s. Still present on 2026-07-28 (15 occurrences in a 30-minute journal window). Non-fatal — the service runs normally. Needs a migration authored by someone who can specify the intended columns; not invented here.
- **The gateway's tracked unit template is behind its installed copy.** Verified 2026-07-28: the two Environment= fixes described under *Gotchas* (HELIX_REDIS_HOST/PORT, HELIX_MODELS_DIR) are present in the installed ~/.config/systemd/user/helixllm-gateway.service but **absent** from the tracked template scripts/systemd/helixllm-gateway.service, which still carries the inert HELIX_CACHE_REDIS_* names. The next scripts/install_systemd_units.sh run therefore silently reverts both fixes, and a fresh clone never had them — the exact loss the *Unit templating* section warns about. Until the template is updated, the running platform and the repository disagree:

```
# shows the drift
diff <(sed "s|@HELIX_ROOT@|$PWD|g; s|@HELIXLLM_BIN@|
    $HOME/.local/bin/helixllm|g" \
    scripts/systemd/helixllm-gateway.service) \
    ~/.config/systemd/user/helixllm-gateway.service
```

- **helixllm-gateway lists no models** — GET /v1/models returns {"object": "list", "data": null} (re-confirmed 2026-07-28) until provider API keys are configured in .env. The API itself works; use /internal/health for readiness.
- **QUIC receive-buffer warning** from helixllm-gateway (failed to sufficiently increase receive buffer size) — a host sysctl tuning matter (net.core.rmem_max), not a service defect. Carried over from an earlier captured run; **not re-observed in the 2026-07-28 pass**, so treat its current presence as unconfirmed.

Resolved since Revision 1

- **LLMsVerifier now publishes real scores.** Revision 1 recorded GET :8100/api/scores returning {"scores": {}} (“nothing measured yet”). As of 2026-07-28 it returns {"scores": {"llamacpp":

`{"total":96,"model_count":1}}` — the verifier seeds its providers from the consumer-owned `config/llmsverifier/config.yaml` and has published its first real aggregate, so the gateway can adopt it instead of its static fallback table. Coverage is one provider so far; the remaining providers stay unscored until a verification pass runs with their API keys configured.

Sources verified 2026-07-28 (Revision 2)

PRIMARILY INTERNALLY DERIVED — this document describes THIS repository's own systemd user-unit installation, so its authoritative sources are the tracked unit definitions, the installer, and the consuming binaries' own configuration code. Verified in this pass:

- **The six installed units** at `~/.config/systemd/user/` read directly: `helixcode-infra`, `llmsverifier`, `helixllm-coder`, `helixllm-gateway`, `helixagent`, `helixcode-server`. The `Requires=` / `Wants=` / `WantedBy=helix.target` claims above are quoted from those files.
- **The boot chain**, by listing `default.target.wants/` (contains `helix.target`), `helix.target.wants/` (contains all six services), and `loginctl show-user -p Linger(Linger=yes)`.
- **Env-tag claims**, by `grep -n 'env:'` over `submodules/helix_llm/internal/shared/config/config.go`: `HELIX_MODELS_DIR` default / `models` at line 70; `HELIX_REDIS_HOST` line 121; `HELIX_REDIS_PORT` default 6379 line 122. The `HELIX_CACHE_REDIS_*` names the template still sets appear nowhere in that file.
- **The gateway health route**, by locating its registration at `submodules/helix_llm/internal/server/server.go:167` (`s.engine.GET("/internal/health", ...)`), and confirming `/v1/models` is the *downstream-provider* probe path (`internal/a2a/downstream.go:72`, `internal/mcpgateway/downstream.go:69`).
- **The port map**, against the ports: entries of `compose.helixcode-infra.yml` (including weaviate's second mapping 50051, previously undocumented) and the 11 services it defines.
- **The template drift**, by expanding `scripts/systemd/helixllm-gateway.service` and diffing it against the installed copy.
- **Runtime state**, by read-only HTTP GETs (`/api/scores`, `/internal/health`, two `/health` routes — all 200) and a read-only `journalctl --user` window for the `distributed_locks` gap.

Honest boundary (§11.4.6 / §11.4.99(B)) — three explicit gaps: (1) the upstream systemd manual pages (`systemd.unit`, `systemd.service`, `systemd.target`, `journalctl`) were NOT re-fetched in this pass, so the generic systemd semantics described here rest on the in-repo unit files alone and carry no external-currency

claim; (2) no `systemctl start/stop/restart/enable` was executed in this pass — the platform was already running and was deliberately not mutated, because other work is live on this shared host (§11.4.174), so the enable/disable commands above are documented from the units' [Install] sections and the installer, not from an executed enable cycle; (3) the QUIC receive-buffer warning is carried over from an earlier run and was not re-observed, as flagged in *Known gaps*. Re-verify against the official systemd documentation per the §11.4.99 180-day staleness window before this guide is used as external-facing operator authority.